

Лабораторная работа № 6.

Автоматизация тестирования Node.js REST API с использованием CI

Тема: Настройка GitHub Actions: npm ci, Prisma Client, unit- и integration-тесты

Цель: Настроить автоматический запуск тестов Internet Shop REST API в GitHub Actions после изменений в репозитории.

Дисциплина: Разработка браузерных приложений

Лабораторная работа №6. Автоматизация тестирования Node.js REST API с использованием GitHub Actions

В лабораторной работе необходимо настроить Continuous Integration для проекта Internet Shop REST API. После отправки изменений в GitHub workflow должен автоматически подготовить Node.js 22, установить зависимости, сгенерировать Prisma Client, подготовить тестовую PostgreSQL, применить migrations и запустить Jest/Supertest.

1. Ожидаемый результат

- В репозитории создан файл .github/workflows/ci.yml.
- Workflow запускается при push в main и при pull request в main.
- Используется Node.js 22.
- Зависимости устанавливаются командой npm ci.
- Prisma Client генерируется автоматически.
- Для integration tests используется отдельная временная база shop_test.
- Команда npm test выполняется автоматически.
- Успешный запуск отображается в GitHub Actions со статусом success.

2. Предварительная проверка проекта

Перед настройкой CI убедитесь, что проект корректно тестируется локально. Ошибки локального проекта сначала исправляются локально, а не внутри GitHub Actions.

Проверка Node.js и npm

```
node --version
npm --version
```

Установка зависимостей и запуск тестов

```
npm ci
npx prisma generate
npm test
```

Версия Node.js должна соответствовать учебному проекту: Node.js 22. Команда npm test должна завершиться без failed tests.

3. Проверка package.json

GitHub Actions будет запускать npm test, поэтому в package.json должен существовать script test.

Пример scripts

```
{
  "scripts": {
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand"
  }
}
```

Параметр `--runInBand` запускает Jest tests последовательно. Для учебного проекта с общей test database это упрощает воспроизводимость.

4. Проверка файлов Git

- `package.json` должен быть добавлен в репозиторий.
- `package-lock.json` должен быть добавлен в репозиторий.
- `prisma/schema.prisma` и `prisma/migrations` должны быть добавлены в репозиторий.
- Test files должны находиться в репозитории.
- `.env` с реальными паролями не должен попадать в Git.

Проверка состояния Git

```
git status
```

5. Создание каталога GitHub Actions

Создание каталога workflow в Windows PowerShell

```
New-Item -ItemType Directory -Force .github\workflows
```

В каталоге `.github/workflows` создайте файл `ci.yml`.

6. Базовая структура workflow

Начало `ci.yml`

```
name: Internet Shop CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
```

Workflow будет запускаться при push в main и при создании или обновлении pull request, направленного в main.

7. Environment variables для тестов

Добавьте тестовые переменные на уровень job. Они используются только внутри CI и не должны ссылаться на production database.

Тестовые переменные

```
env:  
  NODE_ENV: test  
  JWT_SECRET: ci-test-secret  
  DATABASE_URL: postgresql://postgres:postgres@localhost:5432/shop_test?schema=public
```

8. Временная PostgreSQL для integration tests

Integration tests могут обращаться к Prisma и PostgreSQL. В workflow создаём отдельную временную службу PostgreSQL с базой shop_test. Студенту не требуется устанавливать PostgreSQL на GitHub runner вручную.

PostgreSQL service

```
services:  
  postgres:  
    image: postgres:17  
    env:  
      POSTGRES_USER: postgres  
      POSTGRES_PASSWORD: postgres  
      POSTGRES_DB: shop_test  
    ports:  
      - 5432:5432  
    options: >-  
      --health-cmd="pg_isready -U postgres -d shop_test"  
      --health-interval=10s  
      --health-timeout=5s  
      --health-retries=5
```

Health check нужен, чтобы job не начинал обращаться к базе до момента готовности PostgreSQL.

9. Получение проекта

Checkout repository

```
steps:  
  - name: Checkout repository  
    uses: actions/checkout@v7
```

Action checkout загружает содержимое репозитория в рабочий каталог runner.

10. Настройка Node.js

Node.js 22

```
- name: Setup Node.js  
  uses: actions/setup-node@v7  
  with:  
    node-version: '22'  
    cache: 'npm'
```

CI использует ту же major-версию Node.js, что и учебный проект.

11. Установка зависимостей

```
npm ci

- name: Install dependencies
  run: npm ci
```

Для CI используем `npm ci`. Команда устанавливает зависимости по `package-lock.json` и обеспечивает воспроизводимую установку.

12. Генерация Prisma Client

```
Prisma Client

- name: Generate Prisma Client
  run: npx prisma generate
```

13. Подготовка схемы test database

```
Применение migrations

- name: Apply database migrations
  run: npx prisma migrate deploy
```

Команда применяет уже созданные migrations из репозитория к временной test database. Production database в лабораторной работе не используется.

14. Автоматический запуск тестов

```
Jest и Supertest

- name: Run tests
  run: npm test
```

Если хотя бы один test завершится ошибкой, `npm test` вернёт ненулевой exit code, а GitHub Actions отметит job как failure.

15. Итоговый файл ci.yml

```
Полный workflow

name: Internet Shop CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    env:
      NODE_ENV: test
      JWT_SECRET: ci-test-secret
```

```
DATABASE_URL: postgresql://postgres:postgres@localhost:5432/shop_test?schema=public

services:
  postgres:
    image: postgres:17
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shop_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd="pg_isready -U postgres -d shop_test"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=5

steps:
  - name: Checkout repository
    uses: actions/checkout@v7

  - name: Setup Node.js
    uses: actions/setup-node@v7
    with:
      node-version: '22'
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Generate Prisma Client
    run: npx prisma generate

  - name: Apply database migrations
    run: npx prisma migrate deploy

  - name: Run tests
    run: npm test
```

16. Отправка workflow в GitHub

Commit и push

```
git add .
git commit -m "Add GitHub Actions CI"
git push
```

После push откройте репозиторий в GitHub и перейдите во вкладку Actions. Найдите workflow Internet Shop CI.

17. Анализ успешного запуска

1. Откройте последний workflow run.
2. Откройте job test.
3. Убедитесь, что Checkout repository завершился success.
4. Убедитесь, что Setup Node.js завершился success.
5. Проверьте Install dependencies.
6. Проверьте Generate Prisma Client.

7. Проверьте Apply database migrations.
8. Откройте logs шага Run tests.
9. Убедитесь, что Jest сообщает об отсутствии failed tests.

18. Эксперимент с ошибкой

Временно измените один assertion в тесте так, чтобы тест гарантированно завершился failure. Например, замените ожидаемый HTTP status 200 на 201 для endpoint, который возвращает 200.

Пример временной ошибки

```
expect(response.status).toBe(201);
```

Отправка ошибочного изменения

```
git add .
git commit -m "Demonstrate CI failure"
git push
```

Откройте новый workflow run и найдите красный step Run tests. Зафиксируйте в отчёте сообщение Jest: expected, received и exit code.

19. Исправление ошибки

Возврат правильного assertion

```
expect(response.status).toBe(200);
```

Повторная отправка

```
git add .
git commit -m "Fix test"
git push
```

Убедитесь, что новый workflow run завершился success. Таким образом видно, что каждый push автоматически запускает одинаковую проверку.

20. Типичные ошибки

Диагностика CI

Ошибка	Что проверить
npm ci failed	package-lock.json существует и соответствует package.json
prisma generate failed	Prisma config и DATABASE_URL доступны
Connection refused	PostgreSQL service, port 5432 и DATABASE_URL
prisma migrate deploy failed	Файлы prisma/migrations добавлены в Git

Ошибка	Что проверить
npm test failed	Первый failed test и stack trace в logs
401 вместо ожидаемого ответа	Authorization header и тестовый JWT

21. Контрольные вопросы

10. Что такое Continuous Integration?
11. Какое событие запускает workflow в созданном ci.yml?
12. Для чего используется GitHub runner?
13. Почему в CI используется npm ci?
14. Зачем нужен package-lock.json?
15. Почему перед тестами выполняется prisma generate?
16. Почему integration tests не должны использовать production database?
17. Что означает статус failure?
18. Где искать причину ошибки workflow?
19. Что произойдёт после следующего git push?

22. Требования к отчёту

20. Цель лабораторной работы.
21. Фрагмент package.json со script test.
22. Полный файл .github/workflows/ci.yml.
23. Скриншот успешного workflow run.
24. Скриншот job test со списком steps.
25. Фрагмент logs шага Run tests.
26. Скриншот специально созданного failed workflow.
27. Краткое объяснение причины failure.
28. Скриншот повторного успешного запуска после исправления.
29. Ответы на контрольные вопросы.

23. Критерии выполнения

Критерии

Критерий	Результат
Workflow находится в .github/workflows/ci.yml	Обязательно
Используется Node.js 22 и npm ci	Обязательно
Выполняется prisma generate	Обязательно
Подготовлена отдельная test database	Обязательно
Применяются migrations	Обязательно
Jest/Supertest запускаются командой npm test	Обязательно
Продемонстрирован success run	Обязательно
Продемонстрирован и объяснён failure run	Обязательно
После исправления CI снова получает success	Обязательно

24. Вывод

В результате лабораторной работы проект Internet Shop получил базовый CI pipeline. Теперь изменения после git push автоматически проверяются в независимом окружении: устанавливаются зависимости, создаётся Prisma Client, подготавливается test database и запускаются Jest/Supertest. CI становится автоматической контрольной точкой перед дальнейшим развёртыванием приложения.